

Complete Android Build Setup Using Expo Prebuild + EAS Build (No Android SDK Needed)

◆ 1. Prerequisites

You only need **Node.js**, **Expo CLI**, and **EAS CLI**.

Install globally:

```
npm install -g expo-cli eas-cli
```

⚠ No Android Studio, SDK, or Java is required locally. Everything builds in Expo's cloud.

◆ 2. Create a New Expo Project

```
npx create-expo-app myApp  
cd myApp
```

Explanation: - Creates a clean React Native app with Expo's base template. - Contains JS code and configuration (`App.js` , `package.json` , `app.json`). - Starts as a **Managed Workflow** app (no native folders yet).

◆ 3. Run the App in Development Mode

You can run the app locally during development without building an APK.

Start the development server:

```
npx expo start
```

Explanation: - Launches Metro bundler, which serves your JavaScript bundle. - A QR code appears in the terminal.

Test the app on a physical device:

1. Install **Expo Go** from Google Play on your Android phone.

2. Open Expo Go → Scan the QR code.
3. The app loads instantly using your live JS code.

 Every code change auto-refreshes the app. This is your fast development loop before doing a full cloud build.

Optional: Run on an emulator

If you have an Android emulator installed:

```
npx expo start --android
```

This command opens the app in your Android emulator directly.

◆ 4. Convert to Bare Workflow (Native Ready)

```
npx expo prebuild
```

Explanation: - Generates `/android` and `/ios` directories. - `/android` folder holds native Gradle project for Android builds. - Only includes native code for libraries you've installed (not all Expo modules).

You can now build your Android app without Android SDK locally.

◆ 5. Add Only Needed Libraries

Example:

```
expo install react-native-reanimated react-native-gesture-handler
```

Explanation: - Installs compatible native libraries. - During prebuild or EAS build, only these are compiled natively (keeping app lightweight).

◆ 6. Configure EAS Build

Your current `eas.json` looks fine; it defines three profiles: `development`, `preview`, and `production`. You can use these by running commands such as `npx eas build -p android --profile development` to create a development client (with local debugging capabilities using `expo run:android` or Expo Go). The `preview` profile is for internal testing builds, and `production` for final store-ready builds.

```
npx eas build:configure
```

When asked:

Would you like to automatically create an EAS project for @your-username/myApp?

 Choose **Yes**. It links your local app to your Expo account for cloud builds.

Explanation: - Sets up cloud build settings. - Creates an `eas.json` file defining build profiles.

Example:

```
{
  "build": {
    "preview": {
      "distribution": "internal",
      "android": { "buildType": "apk" }
    },
    "production": {
      "distribution": "store",
      "android": { "buildType": "aab" }
    }
  }
}
```

- `apk` = installable test build
- `aab` = Google Play Store upload format

◆ 7. Login to Expo

```
npx expo login
```

Explanation: - Authenticates your CLI with your Expo account. - Required for uploading code and tracking builds on expo.dev.

◆ 8. Build Android App in Cloud

When prompted 'Generate a new Android Keystore? (Y/n)', choose **Y (Yes)** if this is your first time building with EAS for this app. EAS will create and securely store a new keystore in your Expo account automatically. You only need to generate it once per project; afterward it reuses the same signing key for future builds.

For Testing (APK):

```
npx eas build -p android --profile preview
```

Explanation: - Uploads project to Expo's build servers. - Compiles using full Android SDK remotely. - Returns a public link to download `.apk`.

For Play Store (AAB):

```
npx eas build -p android --profile production
```

Explanation: - Builds `.aab` package ready for Play Console upload. - Expo automatically handles signing keys.

◆ 9. Configure App Info

In `app.json`:

```
{
  "expo": {
    "name": "My Android App",
    "slug": "myApp",
    "android": {
      "package": "com.triostack.myapp",
      "versionCode": 1,
      "icon": "./assets/icon.png"
    }
  }
}
```

Explanation: - `package` = unique identifier for Play Store. - `versionCode` = must increment for every release.

◆ 10. Test or Share APK

After build completes, download from the link shown:

✓ Build finished → <https://expo.dev/accounts/<username>/projects/myApp/builds/xxxx>

Install manually or share the link with testers.

You can also use Expo's internal distribution to share pre-release versions easily.

◆ 11. Push Instant JS Updates (No Rebuild Needed)

```
npx expo publish
```

Explanation: - Publishes JavaScript updates OTA (Over-The-Air). - Sends latest JS bundle to users instantly without rebuilding.

◆ 12. Summary

Step	Command	Purpose
1	<code>npx create-expo-app myApp</code>	Create new project
2	<code>npx expo start</code>	Run in dev mode locally
3	<code>npx expo prebuild</code>	Generate native folders
4	<code>expo install <lib></code>	Add native-compatible libraries
5	<code>npx eas build:configure</code>	Configure cloud build
6	<code>npx expo login</code>	Authenticate Expo account
7	<code>npx eas build -p android</code>	Cloud build for APK/AAB
8	<code>npx expo publish</code>	Send instant updates

Result

- Full native Android app built via cloud (no SDK).
- Fast local development using Expo Go.

- Only required libraries compiled.
 - APK/AAB ready for install or Play Store submission.
-

References

- Dashboard: <https://expo.dev>
 - EAS Docs: <https://docs.expo.dev/build>
 - Android Publishing: <https://developer.android.com/studio/publish>
-

End Result:

A fully native Android app, built entirely in the cloud with zero local SDK setup, lightweight prebuild structure, fast dev mode with Expo Go, and instant deploy capabilities.